

Optimizing neural networks

Usman Roshan

How do we find the min value of a function?

- Given $f(x)$ find x that minimizes $f(x)$. This is a key fundamental problem with broad applications across different areas
- Let us start with $f(x)$ that is non-differentiable. For example the objective of traveling salesman problem is non-differentiable.

Local search

- Local search is a fundamental search method in machine learning and AI
- Given a non-differentiable objective we perform local search to find its minimum
- If the objective is differentiable we get the optimal search direction with the gradient

Neural network objective

- Non-linear objective, multiple local minima
- As a result optimization is much harder than that of a convex objective
- Standard approach: gradient descent:
 - Calculate first derivatives of each hidden variable
 - For inner layers we use the chain rule (see google sheet for derivations)

Gradient descent

- So we run gradient descent until convergence, then what is the problem?
- May converge on a local minima and require random restarts
- Overfitting: a big problem for many years
- How can we prevent overfitting?
- How can we explore the search space better without getting stuck in local minima?

Stochastic gradient descent

- A simple but beautifully powerful idea introduced by Leon Bottou in 2000
- Original SGD:
 - While not converged:
 - Select a single datapoint in order from the data
 - Compute gradient with just one point
 - Update parameters
- Pros: broader search
- Cons: final solution may be poor, may be hard to converge

Stochastic gradient descent

- Mini-batch SGD:
 - While not converged:
 - Select a random batch of datapoints
 - Compute gradient with the batch
 - Update parameters
- Mini-batch pros: generally better solution with better convergence than single datapoint
- Batch sizes are usually small

Learning rate

- Key to the search is the step size
- Ideally we start with a somewhat large size (0.1 or 0.01) and reduce by power of 10 after a few epochs
- Adaptive step size is the best but may slow the search

Dropout

- A simple method introduced in 2014 to prevent overfitting
- Procedure:
 - During training we decide with probability p to update a node's weights or not.
 - We set p to be typically 0.5
- Highly effective in deep learning:
 - Decreases overfitting
 - Increases training time
- Can be loosely interpreted as ensemble of networks